

da y . La distanza massima che si ottiene da questa seconda visita è il diametro k dell'albero. L'algoritmo restituisce quindi altezza $\lceil k/2 \rceil$.

Il problema di questo approccio è che soffre dello stesso problema della visita a partire da tutti i nodi: può selezionare un diametro che corrisponde ad un albero generale, non a un albero binario. Si prenda il grafo mostrato sopra: partendo da un nodo a caso, si arriva ad uno dei nodi con grado 1 come nodi a distanza massima. Partendo da uno di questi nodi per la seconda visita, si ottiene un diametro pari a 2. Quindi l'algoritmo restituisce 1, che è sbagliato.

Nonostante l'algoritmo non sia corretto, ho deciso di dare 110% a chi ha proposto questa soluzione, perché come sopra, dimostra la conoscenza di concetti più avanzati; inoltre, la specifica "albero binario" era stata creata per complicare l'algoritmo $O(n^2)$, senza considerare che avrebbe complicato anche questo algoritmo.

Esercizio B1 – Griglia lineare – Punti ≥ 8

Questo esercizio è simile ad altri dati nel passato (ad esempio, il numero di modi per scendere una scala del 12/1/2015, che dà origine al calcolo dei numeri di Tetranacci).

È sufficiente adattare quell'algoritmo, tenendo conto che ora s è un parametro e non un valore fisso. Il problema si risolve semplicemente utilizzando la programmazione dinamica. Sia $T(n)$ il numero di modi in cui è possibile riempire una griglia $1 \times n$; allora $T(n)$ può essere espresso nel modo seguente:

$$T(n) = \begin{cases} 1 & n = 0 \\ \sum_{k=1}^{\min\{n,s\}} T(n-k) & n > 0 \end{cases}$$

Un algoritmo basato su programmazione dinamica per risolvere il problema è il seguente:

```

int count(int n, int s)


---


    int[] T = new int[0...n]
    T[0] = 1
    for i = 1 to n do
        T[i] = 0
        for k = 1 to min(i, s) do
            T[i] = T[i] + T[i - k]
    return T[n]

```

La complessità della procedura è pari a $O(n \cdot s)$, che deriva dai due cicli for annidati.

Esercizio B2 – Tape – Punti ≥ 10

Entrambe le strategie non sono ottimali. Per dimostrarlo, dobbiamo trovare dei controesempi; se possibile, conviene sempre cercare controesempi piccoli, ad esempio di dimensione $n = 2$, perché semplifica i calcoli e permette di individuare delle situazioni "estreme".

Ordine decrescente di probabilità di utilizzo: Sia $p = [0.6, 0.4]$ e sia $d = [200, 100]$. I valori sono già ordinati per probabilità decrescente, il costo in questo caso è

$$C = \frac{0.6 \cdot 200 + 0.4 \cdot 300}{2} = \frac{120 + 120}{2} = 120$$

Se ordiniamo i file in ordine inverso, otteniamo

$$C' = \frac{0.4 \cdot 100 + 0.6 \cdot 300}{2} = \frac{40 + 180}{2} = 110$$

che è un costo inferiore e quindi l'algoritmo greedy proposto non è corretto.

Ordine crescente di lunghezza: Sia $p = [0.1, 0.9]$ e sia $d = [100, 200]$. I valori sono già ordinati per lunghezza crescente; il costo in questo caso è:

$$C = \frac{0.1 \cdot 100 + 0.9 \cdot 300}{2} = \frac{10 + 270}{2} = 140$$

Se ordiniamo i file in ordine inverso, otteniamo

$$C' = \frac{0.9 \cdot 200 + 0.1 \cdot 300}{2} = \frac{180 + 30}{2} = 105$$

che è un costo inferiore e quindi l'algoritmo greedy proposto non è corretto.

Esercizio B3 – Cardinalità – Punti ≥ 12

Il problema può essere risolto tramite la tecnica del backtrack. La funzione principale `minCardinality()` crea il vettore di appoggio e seleziona il primo elemento, che determinerà la cardinalità dell'insieme. Chiama poi la funzione ricorsiva `mcRec()`, che prende in input il vettore S , il valore massimo n e il numero di elementi inseriti finora nel vettore (ossia, l'indice dell'ultimo elemento inserito).

Nella funzione ricorsiva, ci sono due casi:

- se il numero di elementi inseriti è uguale al valore minimo, abbiamo raggiunto la cardinalità richiesta e possiamo stampare il vettore;
- altrimenti, si esegue un ciclo `for`, a partire da $S[i] + 1$ (il numero successivo all'ultimo numero inserito) fino ad n ; all'interno del ciclo, il valore j viene inserito nel vettore e si chiama il prossimo elemento.

Si noti che la dimensione del vettore avrebbe potuto essere n , ma abbiamo scelto il valore $\lceil n/2 \rceil$. Si consideri un insieme il cui valore minore è k e di conseguenza ha dimensione k . I valori possibili con cui riempire questo vettore, compresi fra $k + 1$ e n , sono $n - k$. Dobbiamo selezionare $k - 1$ elementi per riempire vettore (un elemento è già preso, ed è k), quindi $k - 1$ deve essere minore o uguale a $n - k$, altrimenti non avrò abbastanza valori da scegliere. Quindi:

$$k - 1 \leq n - k \quad \Leftrightarrow \quad 2k \leq n + 1 \quad \Leftrightarrow \quad k \leq \frac{n + 1}{2} \quad \Leftrightarrow \quad k \leq \lceil n/2 \rceil$$

`minCardinality(int n)`

```
int[] = new int[1...⌈n/2⌉]
for i = 1 to n do
    S[1] = i
    mcRec(S, n, 2)
```

`mcRec(int[] S, int n, int i)`

```
if i == S[1] then
    print S[1...S[i]]
else
    for j = S[i] + 1 to n do
        S[i + 1] = j
        mcRec(S, n, i + 1)
```

Analisi complessità, versione semplice, accettabile Per quanto riguarda la complessità, possiamo semplicemente osservare che essendoci chiamate multiple all'interno della chiamata ricorsiva, la complessità è superpolinomiale. Questa analisi è grossolana, perché il numero di chiamate diminuisce al crescere della dimensione dell'insieme.

Analisi complessità, versione dettagliata, non necessaria per il compito Sia k la dimensione del vettore e quindi il valore minimo, necessariamente presente nel vettore; come detto, esistono $n - k$ valori maggiori di k e dobbiamo selezionare $k - 1$ elementi fra di essi, per raggiungere la cardinalità k . Possiamo quindi calcolare il numero $S(n, k)$ di insiemi di dimensione k utilizzando il coefficiente binomiale:

$$S(n, k) = \binom{n - k}{k - 1} = \frac{(n - k)!}{(k - 1)!((n - k) - (k - 1))!} = \frac{(n - k)!}{(k - 1)!(n - 2k + 1)!}$$

Il numero di insiemi è quindi 1 e $\lfloor n/2 \rfloor$.

$$S(n) = \sum_{k=1}^{\lfloor n/2 \rfloor} S(n, k)$$

Considerato che stampare un insieme di dimensione k ha costo k , il costo è pari a:

$$T(n) = \sum_{k=1}^{\lfloor n/2 \rfloor} S(n, k) \cdot k$$

I valori di $S(n)$ per $n = 1 \dots 10$ sono $[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]$, ovvero la successione di Fibonacci, quindi deve esserci un modo diverso (e sperabilmente più semplice) per completare l'analisi che al momento mi sfugge.